

DISCIPOLAT — TRANSFORMATION EN SAAS MULTI-TENANT HIÉRARCHIQUE

PROMPT MAÎTRE POUR AGENT DE DÉVELOPPEMENT

1. MISSION

Tu es chargé de transformer l'application existante **Discipolat** en une véritable plateforme SaaS multi-tenant, multi-église, multi-niveaux d'administration et internationalisable.

Tu travailles obligatoirement sur le code existant.

Tu dois :

- comprendre l'architecture actuelle ;
- préserver les fonctionnalités existantes ;
- identifier les éléments déjà implémentés ;
- éviter les doublons ;
- migrer progressivement les données existantes ;
- renforcer la sécurité ;
- implémenter le multi-tenant de manière native ;
- rendre toutes les fonctionnalités existantes tenant-aware ;
- permettre la création d'églises, sous-églises, campus, régions, départements et groupes ;
- mettre en place une administration hiérarchique complète ;
- rendre l'ensemble compatible Web + Flutter Mobile.

Le résultat doit être suffisamment robuste pour supporter plusieurs milliers d'organisations.

2. RÈGLE ABSOLUE : NE PAS CODER IMMÉDIATEMENT

Avant toute modification :

1. inspecter le repository ;

2. comprendre le backend ;
3. comprendre PostgreSQL ;
4. lire toutes les migrations Flyway ;
5. analyser les entités ;
6. analyser les repositories ;
7. analyser les services ;
8. analyser les controllers ;
9. analyser Spring Security ;
10. analyser JWT ;
11. analyser les rôles existants ;
12. analyser les permissions existantes ;
13. analyser le frontend React ;
14. analyser l'application Flutter ;
15. analyser Drift/SQLite ;
16. analyser Redis ;
17. analyser WebSocket ;
18. analyser les notifications ;
19. analyser les fichiers ;
20. analyser les tests.

Ne modifie aucun code pendant cette première analyse.

Produis d'abord :

MULTI-TENANT & ADMINISTRATION ARCHITECTURE AUDIT

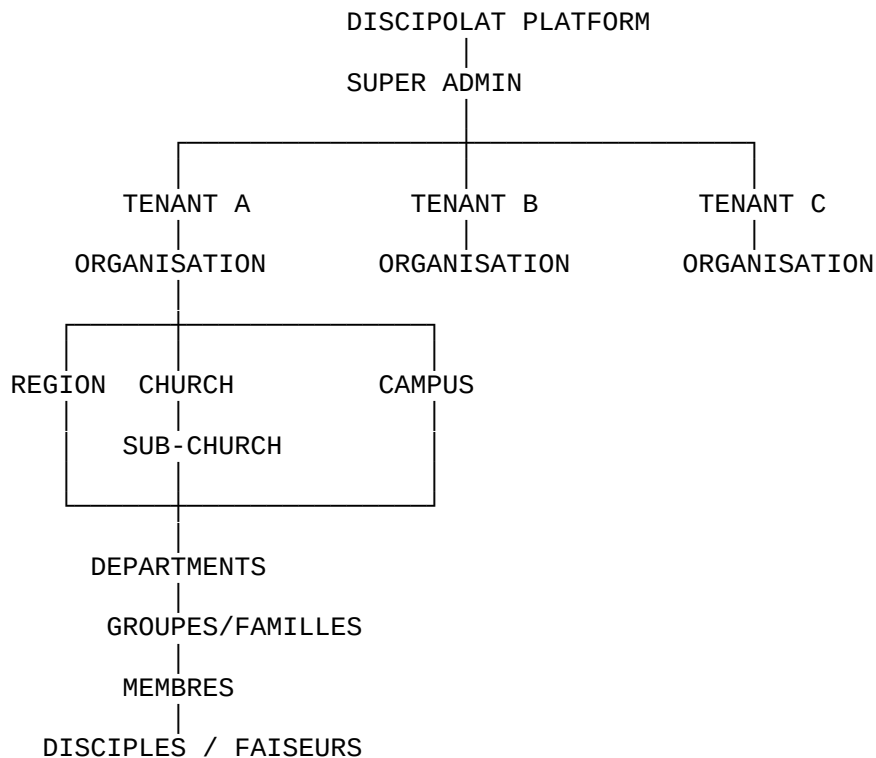
Le rapport doit identifier :

- ce qui existe ;
- ce qui peut être réutilisé ;
- ce qui doit être modifié ;
- ce qui doit être supprimé ;

- les risques de sécurité ;
- les risques IDOR ;
- les risques cross-tenant ;
- les risques liés aux rôles ;
- les risques liés au cache ;
- les risques liés aux fichiers ;
- les risques liés au mobile ;
- les risques liés aux WebSockets ;
- la stratégie de migration ;
- l'architecture cible.

3. VISION ARCHITECTURALE

Discipolat doit devenir :



Le système doit supporter différentes structures organisationnelles.

Une organisation peut être :

- une église indépendante ;

- une église avec plusieurs campus ;
 - un réseau d'églises ;
 - une dénomination ;
 - une mission ;
 - une organisation chrétienne ;
 - une communauté internationale.
-

4. LES NIVEAUX D'ADMINISTRATION

Implémenter une hiérarchie complète.

NIVEAU 0 — PLATFORM SUPER ADMIN

Rôle :

PLATFORM_SUPER_ADMIN

C'est l'administrateur de la plateforme Discipolat.

Il n'appartient pas à un tenant classique.

Il administre l'infrastructure SaaS.

Il peut :

- créer un tenant ;
- suspendre un tenant ;
- réactiver un tenant ;
- archiver un tenant ;
- gérer les plans ;
- gérer les abonnements ;
- gérer les quotas ;
- gérer les fonctionnalités disponibles ;
- gérer les feature flags ;
- gérer les pays ;
- gérer les devises ;
- gérer les langues ;

- gérer les paramètres globaux ;
- consulter les métriques globales ;
- gérer les administrateurs de plateforme ;
- gérer les tickets/support ;
- consulter les logs plateforme ;
- gérer les versions ;
- gérer les incidents ;
- gérer la configuration globale.

IMPORTANT :

Le Super Admin ne doit pas avoir un accès silencieux aux données privées des tenants.

Toute consultation sensible doit être :

- autorisée ;
 - justifiée ;
 - auditée ;
 - traçable.
-

5. PLATFORM SUPPORT ADMIN

Créer éventuellement un rôle :

PLATFORM_SUPPORT

Il peut :

- consulter les informations techniques nécessaires ;
- diagnostiquer un tenant ;
- consulter les erreurs ;
- consulter l'état des services ;
- aider les administrateurs ;
- utiliser une impersonation contrôlée.

Il ne doit PAS automatiquement avoir accès à toutes les données métier.

Séparer les permissions techniques des permissions de données.

6. PLATFORM BILLING ADMIN

Créer :

PLATFORM_BILLING_ADMIN

Il gère :

- plans ;
- abonnements ;
- factures ;
- paiements SaaS ;
- coupons ;
- quotas ;
- usage ;
- facturation.

Il ne doit pas avoir accès aux données pastorales des utilisateurs.

7. TENANT OWNER

Chaque tenant possède un propriétaire :

TENANT_OWNER

Il représente le propriétaire principal de l'organisation.

Il peut :

- configurer le tenant ;
- créer l'église principale ;
- créer les sous-églises ;
- créer les campus ;
- créer les régions ;
- créer les départements ;
- gérer les administrateurs ;
- gérer les rôles ;
- gérer les permissions ;

- configurer le branding ;
- activer/désactiver les modules ;
- gérer les paramètres ;
- gérer les membres ;
- gérer les parcours ;
- consulter les statistiques du tenant.

Il peut déléguer des permissions.

Il peut créer d'autres administrateurs.

Il ne peut PAS accéder à un autre tenant.

8. TENANT ADMIN

Rôle :

TENANT_ADMIN

Il possède des permissions d'administration du tenant mais n'est pas nécessairement propriétaire.

Il peut gérer selon les permissions :

- utilisateurs ;
- églises ;
- sous-églises ;
- campus ;
- départements ;
- familles ;
- groupes ;
- parcours ;
- formations ;
- événements ;
- notifications ;
- paramètres.

Il ne doit pas pouvoir :

- supprimer le tenant ;

- transférer arbitrairement la propriété ;
 - modifier les paramètres plateforme ;
 - accéder à d'autres tenants.
-

9. ORGANIZATION ADMIN

Créer un niveau permettant de gérer l'organisation sans donner les droits plateforme.

ORGANIZATION_ADMIN

Il peut gérer :

- structure ;
- utilisateurs ;
- membres ;
- responsables ;
- départements ;
- groupes ;
- parcours.

Son périmètre est le tenant.

10. REGION ADMIN

Si une organisation possède des régions :

REGION_ADMIN

Il peut gérer uniquement :

REGION

```
├── Church A
├── Church B
└── Church C
```

Il peut :

- voir les statistiques de sa région ;
- gérer les responsables ;
- consulter les églises ;

- superviser les parcours ;
- consulter les rapports ;
- gérer certaines ressources.

Il ne peut pas administrer :

- une autre région ;
 - le tenant entier ;
 - les données privées hors périmètre.
-

11. CHURCH ADMIN

Rôle :

CHURCH_ADMIN

Il administre une église précise.

Il peut :

- gérer les membres de son église ;
- gérer les familles ;
- gérer les départements ;
- gérer les groupes ;
- gérer les responsables ;
- gérer les événements ;
- gérer les parcours ;
- consulter les rapports ;
- gérer les notifications locales.

Il ne peut pas voir une autre église du même tenant si son scope ne l'autorise pas.

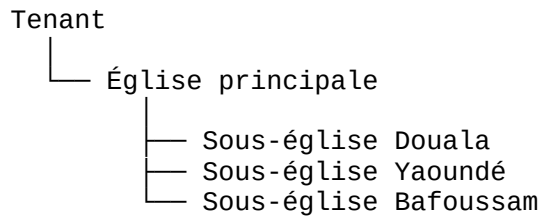
12. SUB-CHURCH ADMIN

Rôle :

SUB_CHURCH_ADMIN

Il administre uniquement une sous-église.

Exemple :



L'administrateur de Douala ne doit pas pouvoir accéder aux membres de Yaoundé.

13. CAMPUS ADMIN

Rôle :

CAMPUS_ADMIN

Il administre un campus précis.

Même logique d'isolation.

14. DEPARTMENT ADMIN

Rôle :

DEPARTMENT_ADMIN

Exemples :

- Jeunesse ;
- Femmes ;
- Hommes ;
- Enfants ;
- Communication ;
- Intercession ;
- Louange ;
- Évangélisation ;
- Technique.

Le responsable ne voit que les ressources autorisées de son département.

15. DEPARTMENT LEADER

DEPARTMENT_LEADER

Il peut :

- gérer son équipe ;
 - consulter les membres de son périmètre ;
 - créer des activités ;
 - consulter les présences ;
 - suivre les objectifs ;
 - produire les rapports.
-

16. FAMILY LEADER

FAMILY_LEADER

Il gère sa famille/groupe.

Il peut :

- consulter les membres ;
- enregistrer les visites ;
- créer des rapports ;
- suivre les besoins ;
- organiser les rencontres ;
- suivre les parcours.

Il ne peut pas consulter les autres familles.

17. DISCIPLE MAKER / FAISEUR DE DISCIPLE

DISCIPLE_MAKER

Il peut :

- gérer les personnes qui lui sont attribuées ;

- créer des rapports ;
- enregistrer des visites ;
- suivre les parcours ;
- créer des notes autorisées ;
- consulter les tâches ;
- recevoir des recommandations IA autorisées.

Il ne peut pas voir toute l'église par défaut.

18. MENTOR

MENTOR

Il accède uniquement aux personnes qui lui sont attribuées.

Il peut :

- suivre les objectifs ;
 - accompagner ;
 - commenter ;
 - proposer des ressources ;
 - suivre la progression.
-

19. MEMBER

MEMBER

Un membre peut :

- consulter son profil ;
- modifier ses informations autorisées ;
- consulter son parcours ;
- suivre ses formations ;
- consulter ses événements ;
- envoyer des demandes ;
- consulter ses notifications ;

- gérer ses préférences ;
- consulter ses données personnelles.

Il ne peut pas accéder aux données internes des autres membres.

20. GUEST

GUEST

Accès limité :

- contenus publics ;
- événements publics ;
- informations publiques ;
- inscription.

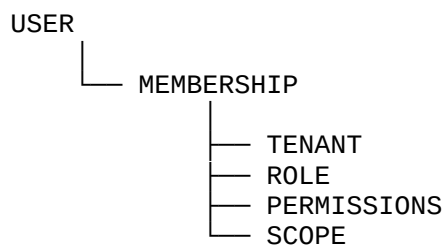
Aucune donnée privée.

21. ARCHITECTURE RBAC + SCOPE

NE PAS construire uniquement un système :

ROLE = ADMIN

Créer :



Une membership doit permettre de savoir :

user_id
tenant_id
role_id
scope_type
scope_id
status
created_at
updated_at

22. PERMISSIONS

Créer un système de permissions atomiques.

Exemples :

TENANT_VIEW
TENANT_UPDATE
TENANT_DELETE

USER_VIEW
USER_CREATE
USER_UPDATE
USER_DELETE

MEMBER_VIEW
MEMBER_CREATE
MEMBER_UPDATE
MEMBER_DELETE

CHURCH_VIEW
CHURCH_CREATE
CHURCH_UPDATE
CHURCH_DELETE

SUB_CHURCH_VIEW
SUB_CHURCH_CREATE
SUB_CHURCH_UPDATE
SUB_CHURCH_DELETE

CAMPUS_VIEW
CAMPUS_CREATE
CAMPUS_UPDATE
CAMPUS_DELETE

DEPARTMENT_VIEW
DEPARTMENT_CREATE
DEPARTMENT_UPDATE
DEPARTMENT_DELETE

FAMILY_VIEW
FAMILY_CREATE
FAMILY_UPDATE
FAMILY_DELETE

REPORT_VIEW
REPORT_CREATE
REPORT_UPDATE

EVENT_VIEW
EVENT_CREATE
EVENT_UPDATE
EVENT_DELETE

COURSE_VIEW
COURSE_CREATE
COURSE_UPDATE
COURSE_DELETE

FINANCE_VIEW
FINANCE_MANAGE

AI_USE
AI_ADMIN

SETTINGS_VIEW
SETTINGS_UPDATE

AUDIT_VIEW

EXPORT_DATA

INVITE_USER
MANAGE_ROLES
MANAGE_PERMISSIONS

23. SCOPE DES PERMISSIONS

Une permission doit pouvoir avoir une portée.

Types :

PLATFORM
TENANT
REGION
CHURCH
SUB_CHURCH
CAMPUS
DEPARTMENT
FAMILY
OWN
ASSIGNED

Exemple :

MEMBER_VIEW
scope = DEPARTMENT
scope_id = 15

signifie :

l'utilisateur peut voir uniquement les membres du département 15.

24. PERMISSIONS PERSONNALISÉES PAR TENANT

Un tenant doit pouvoir créer ses propres rôles.

Exemple :

ROLE :
Coordinateur Jeunesse

Permissions :
MEMBER_VIEW
EVENT_CREATE
REPORT_CREATE
COURSE_VIEW

Le système ne doit pas hardcoder toutes les possibilités.

Prévoir :

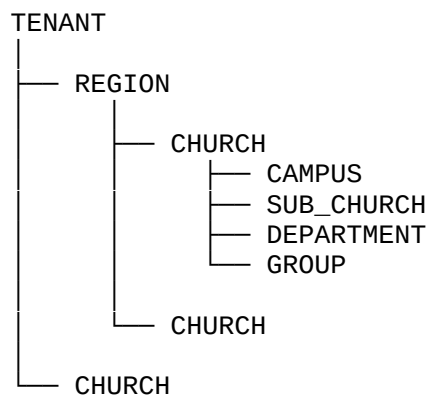
Role
Permission
RolePermission
TenantRole

avec distinction entre rôles système et rôles personnalisés.

25. HIÉRARCHIE ORGANISATIONNELLE

Créer une structure flexible.

Exemple :



Utiliser une structure adaptée au modèle existant.

Ne pas créer plusieurs tables redondantes si une architecture générique peut être utilisée proprement.

26. ORGANIZATION NODE

Si pertinent après audit, créer un modèle :

OrganizationNode

Champs :

id
tenant_id
parent_id
type
name
code
slug
status
level
path
country
city
timezone
metadata
created_at
updated_at

Types :

ROOT
REGION
DISTRICT
CHURCH
SUB_CHURCH
CAMPUS
ASSEMBLY
DEPARTMENT
GROUP

L'architecture doit rester extensible.

27. TENANT SETTINGS

Chaque tenant possède sa configuration.

Exemples :

name
slug
description
logo
favicon
primaryColor
secondaryColor
language
timezone
country
currency
dateFormat
phoneCountryCode
email
phone
website

28. BRANDING

Chaque tenant doit pouvoir personnaliser son environnement.

Exemple :

Tenant A :

Logo : logo-a.png

Primary : #...

Secondary : #...

Tenant B :

Logo : logo-b.png

Primary : #...

Secondary : #...

Le frontend doit charger dynamiquement le branding.

29. MODULES PAR TENANT

Créer :

TenantFeature

Exemples :

DISCIPLES

ACADEMY

AI

FINANCE

EVENTS

CHAT

NOTIFICATIONS

MARKETPLACE

WHATSAPP

MOBILE_MONEY

ANALYTICS

Un tenant peut activer/désactiver les modules.

IMPORTANT :

Désactiver visuellement une fonctionnalité n'est PAS suffisant.

Le backend doit également refuser son utilisation.

30. PLANS SAAS

Prévoir :

FREE
STARTER
PRO
ENTERPRISE

Chaque plan possède :

- utilisateurs maximum ;
- églises maximum ;
- sous-églises maximum ;
- stockage ;
- IA ;
- Academy ;
- finance ;
- API ;
- support ;
- fonctionnalités disponibles.

Ne jamais mettre ces limites directement dans les controllers.

Créer un service :

SubscriptionService
QuotaService
FeatureAccessService

31. QUOTAS

Prévoir :

MAX_USERS
MAX_CHURCHES
MAX_SUB_CHURCHES
MAX_CAMPUSES
MAX_STORAGE
MAX_AI_REQUESTS
MAX_ADMINS
MAX_COURSES
MAX_EVENTS

Les quotas doivent être vérifiés côté backend.

32. TENANT CONTEXT

Créer un mécanisme central :

TenantContext

Il doit identifier le tenant courant à partir de l'utilisateur authentifié et de sa membership.

NE JAMAIS faire confiance aveuglément à :

tenantId
churchId
userId

fourni par le frontend.

33. MULTI-TENANT USER

Un utilisateur doit pouvoir appartenir à plusieurs tenants.

Exemple :

USER
Jean

Memberships :

Tenant A
ROLE = TENANT_ADMIN

Tenant B
ROLE = MEMBER

Tenant C
ROLE = DISCIPLE_MAKER

Il doit rester un seul compte utilisateur.

34. TENANT SWITCHER

Si l'utilisateur appartient à plusieurs tenants :

Choisir votre organisation

Église de la Grâce
Mission Internationale
Réseau Afrique

Après sélection :

ACTIVE TENANT

Toutes les données doivent être rechargées avec ce contexte.

35. ORGANIZATION SWITCHER

Si un utilisateur possède plusieurs périmètres :

Église principale
Campus Douala
Jeunesse

permettre de changer de contexte selon ses permissions.

36. MOBILE FLUTTER

Le mobile doit devenir totalement multi-tenant.

Prévoir :

TenantSession
ActiveTenant
ActiveOrganizationNode
Membership
Permissions

Le tenant actif doit être clairement visible.

Le cache local doit être isolé.

37. OFFLINE

Toutes les données offline doivent être tenant-aware.

Interdit :

member : 123

Préférer :

tenant:A:member:123

ou une structure équivalente.

Lors d'un changement de tenant :

- nettoyer les caches incompatibles ;
 - recharger le contexte ;
 - vérifier les permissions ;
 - synchroniser uniquement les données autorisées.
-

38. REDIS

Toutes les clés sensibles doivent être tenant-aware.

Exemple :

```
tenant:{tenantId}:user:{userId}
tenant:{tenantId}:permissions:{userId}
tenant:{tenantId}:dashboard:{userId}
```

Auditer toutes les clés Redis existantes.

39. FILE STORAGE

Les fichiers doivent être isolés :

```
tenants/
  {tenantId}/
    organizations/
    users/
    courses/
    reports/
    documents/
```

Aucun utilisateur ne doit pouvoir télécharger un fichier d'un autre tenant en connaissant son ID.

40. WEBSOCKET

Les channels doivent être isolés :

```
tenant:A:notifications
tenant:A:chat
```

tenant:B:notifications
tenant:B:chat

Tester explicitement les tentatives de cross-tenant.

41. NOTIFICATIONS

Toutes les notifications doivent posséder le bon contexte :

tenantId
recipientId
organizationNodeId

Un utilisateur multi-tenant ne doit jamais recevoir une notification dans le mauvais contexte.

42. AUDIT LOG

Créer un système d'audit robuste.

Enregistrer les actions sensibles :

LOGIN
TENANT_CREATED
TENANT_UPDATED
USER_CREATED
ROLE_CHANGED
PERMISSION_CHANGED
CHURCH_CREATED
SUB_CHURCH_CREATED
EXPORT
DELETE
IMPERSONATION
PAYMENT
SETTINGS_CHANGED

Champs :

id
actorId
tenantId
organizationNodeId
action
resourceType
resourceId
timestamp
result
ip
userAgent
metadata

Ne pas stocker inutilement de données sensibles dans les logs.

43. IMPERSONATION

Le Super Admin doit pouvoir entrer temporairement dans un tenant pour support.

Workflow :

```
Super Admin
  ↓
Sélectionne Tenant
  ↓
Indique raison
  ↓
Confirmation
  ↓
Session temporaire
  ↓
Tenant
```

Afficher clairement :

⚠ MODE SUPPORT
Vous consultez actuellement le tenant X

Toute action est auditée.

Prévoir expiration automatique.

44. SECURITY — IDOR

Créer des tests automatisés.

Exemple :

Tenant A possède :

`member/100`

Utilisateur du Tenant B :

`GET /members/100`

Résultat :

403

ou :

404

selon la stratégie choisie.

Jamais :

200

Tester également :

- users ;
- churches ;
- sub-churches ;
- families ;
- reports ;
- events ;
- courses ;
- files ;
- notifications ;
- payments ;
- messages ;
- audit logs.

45. SECURITY MATRIX

Créer une matrice de permissions.

Exemple :

Niveau	Tenant	Church	SubChurch	Department	Family	Member
Super Admin	✓	selon support	selon support	selon support	selon support	selon support
Tenant Owner	✓	✓	✓	✓	✓	✓
Tenant Admin	✓	✓	✓	✓	✓	✓
Region Admin	limité	région	région	région	région	région
Church Admin	✗ global	✓	selon scope	selon scope	selon scope	selon scope
SubChurch Admin	✗	✗	✓	✓	✓	✓
Department Admin	✗	✗	✗	✓	selon scope	selon scope
Family Leader	✗	✗	✗	✗	✓	membres famille

Niveau	Tenant	Church	SubChurch	Department	Family	Member
Disciple Maker	×	×	×	×	assigné	assignés
Member	×	×	×	×	limité	soi-même

Cette matrice doit être implémentée réellement côté backend.

46. DATA ACCESS POLICY

Créer si pertinent une couche :

`AuthorizationService`

ou équivalent.

Elle doit répondre à :

```
canView(user, resource)
canCreate(user, scope)
canUpdate(user, resource)
canDelete(user, resource)
```

Ne pas répéter manuellement une logique incohérente dans tous les controllers.

47. BASE DE DONNÉES

Toutes les données métier tenant-aware doivent avoir une stratégie claire.

Selon l'architecture actuelle, choisir entre :

`tenant_id`

sur les tables métier,

ou une architecture plus avancée.

Pour la première version commerciale :

privilégier une architecture explicite, robuste, facilement testable et performante.

Évaluer PostgreSQL Row Level Security uniquement si elle apporte une vraie sécurité supplémentaire sans rendre l'architecture inutilement complexe.

48. INDEX

Créer les indexes nécessaires.

Exemples :

```
tenant_id  
tenant_id + id  
tenant_id + status  
tenant_id + created_at  
tenant_id + organization_node_id
```

Analyser les requêtes réelles avant de multiplier les indexes.

49. MIGRATION DES DONNÉES EXISTANTES

IMPORTANT.

Le projet contient déjà des données et fonctionnalités.

NE PAS les perdre.

Créer une migration :

```
Existing Data  
↓  
Create Default Tenant  
↓  
Attach existing records  
↓  
Create memberships  
↓  
Create default roles  
↓  
Create organization root  
↓  
Validate integrity
```

Créer une stratégie de rollback/backup.

50. ONBOARDING D'UN TENANT

Créer un parcours professionnel.

```
Créer mon organisation  
↓  
Nom  
↓  
Pays  
↓  
Devise  
↓
```

Timezone
↓
Logo
↓
Structure
↓
Église principale
↓
Sous-églises
↓
Administrateurs
↓
Modules
↓
Import membres
↓
Terminé

51. CRÉATION D'UNE SOUS-ÉGLISE

Un Tenant Owner ou utilisateur autorisé doit pouvoir :

Créer une sous-église

Informations :

Nom
Code
Slug
Type
Parent
Pays
Ville
Timezone
Description
Logo
Responsable

Le responsable reçoit une invitation.

Son scope est automatiquement configuré.

52. INVITATIONS

Créer :

Invitation

avec :

id
tenantId
organizationNodeId
email
role
permissions/scope
tokenHash
expiresAt
status
invitedBy

Workflow :

Invitation
↓
Email
↓
Acceptation
↓
Authentification
↓
Membership
↓
Role
↓
Scope

Le token doit être sécurisé et à usage unique.

53. HÉRITAGE DES CONFIGURATIONS

Prévoir :

DEFAULT
INHERITED
OVERRIDDEN

Exemple :

Tenant :

timezone = Africa/Douala

Sous-église :

timezone = INHERITED

ou :

timezone = Europe/Paris

dans le cas d'une implantation internationale.

54. RESSOURCES GLOBALES / LOCALES

Chaque ressource doit pouvoir être :

TENANT_GLOBAL
ORGANIZATION_LOCAL

Exemples :

Cours global du tenant :

visible dans toutes les églises

Cours local :

visible uniquement dans une sous-église

Même logique pour :

- événements ;
- annonces ;
- formations ;
- documents ;
- parcours ;
- notifications.

55. FRONTEND REACT

Créer :

TenantProvider
OrganizationProvider
PermissionProvider

ou utiliser les abstractions existantes si elles remplissent déjà ce rôle.

Le frontend doit connaître :

currentUser
activeTenant
activeMembership
activeOrganizationNode
rôles
permissions
features
subscription
branding

56. ROUTES PROTÉGÉES

Créer des guards adaptés :

RequireAuthentication
RequireTenant
RequirePermission
RequireScope
RequireFeature

Mais ATTENTION :

Les guards frontend servent uniquement à l'expérience utilisateur.

La sécurité réelle reste côté backend.

57. SUPER ADMIN WEB

Créer une interface séparée :

/platform

avec :

Dashboard
Tenants
Subscriptions
Plans
Users
Feature Flags
Support
Audit
System Health
Settings

58. TENANT ADMIN WEB

Créer :

/admin

avec :

Overview
Organization
Churches
Sub-Churches

Campuses
Departments
Groups
Members
Users
Roles
Permissions
Courses
Events
Notifications
Branding
Settings
Modules
Subscription
Audit

Afficher uniquement ce qui est autorisé.

59. MOBILE ADMINISTRATION

Le mobile ne doit pas nécessairement exposer toutes les fonctions du Super Admin.

Adapter les interfaces au contexte.

Un Church Admin peut gérer :

- membres ;
- familles ;
- groupes ;
- rapports ;
- événements ;
- notifications.

Le Super Admin reste prioritairement web.

60. SEARCH

La recherche doit toujours respecter :

tenant
+
scope
+
permission

Exemple :

un Department Admin recherchant :

Jean

ne doit obtenir que les personnes accessibles dans son périmètre.

61. EXPORT

Les exports doivent être tenant-aware.

Avant export :

authentication
authorization
scope validation
audit

L'utilisateur ne doit exporter que les données qu'il est autorisé à consulter.

62. SUPPRESSION

Évaluer :

- soft delete ;
- archivage ;
- suppression définitive.

Pour les données critiques :

préférer un workflow sécurisé et audité.

Ne jamais permettre à un simple admin local de supprimer arbitrairement des données globales.

63. MULTI-TENANT ET IA

Toutes les futures fonctionnalités IA doivent respecter le tenant.

Un modèle IA ne doit jamais mélanger :

Tenant A data
+
Tenant B data

Le contexte RAG doit être strictement filtré.

Créer une séparation logique :

tenantId
organizationNodeId

pour les documents indexés.

64. MULTI-TENANT ET ACADEMY

Les cours peuvent être :

PLATFORM
TENANT
CHURCH
SUB_CHURCH

Un cours privé d'une organisation ne doit jamais être récupéré par une autre.

65. MULTI-TENANT ET CHAT

Les conversations doivent être liées à :

tenantId
conversationId
participants
scope

Un utilisateur multi-tenant ne doit jamais mélanger les conversations entre organisations.

66. MULTI-TENANT ET PAIEMENTS

Toutes les transactions doivent être liées à :

tenantId
organizationNodeId
currency
provider
transactionReference

Les confirmations doivent être faites côté serveur.

Prévoir idempotence et reconciliation.

67. MULTI-TENANT ET NOTIFICATIONS PUSH

Le token appareil peut être associé à plusieurs contextes.

Ne jamais envoyer une notification d'un tenant à un autre par erreur.

68. MULTI-TENANT ET ANALYTICS

Les statistiques doivent respecter le scope.

Un :

Department Leader

ne voit que les statistiques de son département.

Un :

Church Admin

voit son église.

Un :

Tenant Owner

voit tout le tenant.

Un :

Super Admin

voit les métriques plateforme autorisées.

69. TESTS OBLIGATOIRES

Créer une suite :

MultiTenantSecurityTests

Tester :

Tenant isolation

A → A = OK

A → B = REFUSED

B → A = REFUSED

Role isolation

Member → Admin endpoint = REFUSED

Department Admin → autre département = REFUSED

Church Admin → autre église = REFUSED

Tenant Admin → autre tenant = REFUSED

Super Admin → tenant = contrôlé/audité

Resource isolation

Members

Families

Reports

Events

Courses

Files

Messages

Notifications

Payments

70. TESTS DE NON-RÉGRESSION

Toutes les fonctionnalités actuelles doivent continuer à fonctionner.

Tester notamment :

- authentification ;
- utilisateurs ;
- familles ;
- disciples ;
- faiseurs ;
- rapports ;
- notifications ;
- dashboard ;
- transferts ;

- audit ;
 - etc.
-

71. PERFORMANCE

Tester avec :

1 tenant
10 tenants
100 tenants
1 000 tenants
10 000 tenants

Identifier :

- N+1 ;
 - queries lentes ;
 - indexes manquants ;
 - caches ;
 - pagination ;
 - recherche.
-

72. DOCUMENTATION

Créer/mettre à jour :

docs/MULTI_TENANT_ARCHITECTURE.md
docs/ADMINISTRATION_MODEL.md
docs/RBAC.md
docs/TENANT_SECURITY.md
docs/ORGANIZATION_HIERARCHY.md
docs/TENANT_ONBOARDING.md

Ne pas créer des doublons si des documents équivalents existent.

73. GIT

Faire des commits atomiques.

Exemples :

feat(multitenancy): introduce tenant domain model
feat(security): implement tenant context
feat(auth): implement tenant memberships
feat(auth): implement scoped RBAC
feat(organization): add hierarchical organization nodes
feat(admin): add tenant administration
feat(admin): add platform super admin
feat(mobile): add tenant switching
test(security): add cross tenant isolation tests
fix(multitenancy): prevent cross tenant resource access

Ne jamais faire :

feat: implement everything

74. ORDRE OBLIGATOIRE D'IMPLÉMENTATION

Ne développe pas tout simultanément.

PHASE 1 — AUDIT

Aucun code.

PHASE 2 — CORE TENANT

1. Tenant
 2. Tenant status
 3. Tenant context
 4. Tenant membership
 5. Tenant isolation
 6. migration données existantes
-

PHASE 3 — IDENTITÉ

- 7. Multi-tenant user
 - 8. Tenant switching
 - 9. Invitations
 - 10. Session/context
-

PHASE 4 — RBAC

- 11. Roles
 - 12. Permissions
 - 13. Scoped permissions
 - 14. Custom roles
 - 15. Authorization service
-

PHASE 5 — HIÉRARCHIE

- 16. OrganizationNode
 - 17. Regions
 - 18. Churches
 - 19. Sub-churches
 - 20. Campuses
 - 21. Departments
 - 22. Groups/Families
-

PHASE 6 — ADMINISTRATION

- 23. Super Admin
- 24. Tenant Owner
- 25. Tenant Admin
- 26. Organization Admin
- 27. Region Admin

- 28.Church Admin
 - 29.Sub-Church Admin
 - 30.Campus Admin
 - 31.Department Admin
 - 32.Family Leader
 - 33.Disciple Maker
 - 34.Mentor
 - 35.Member
 - 36.Guest
-

PHASE 7 — CONFIGURATION

- 37.Tenant settings
 - 38.Branding
 - 39.Modules
 - 40.Feature flags
 - 41.Plans
 - 42.Quotas
 - 43.Subscription
-

PHASE 8 — WEB

- 44.Super Admin Dashboard
 - 45.Tenant Admin Dashboard
 - 46.Organization tree
 - 47.Role management
 - 48.Permission management
 - 49.Tenant settings
 - 50.Branding
 - 51.Tenant switcher
-

PHASE 9 — MOBILE

- 52.Tenant selection
 - 53.Tenant switcher
 - 54.Organization switcher
 - 55.Offline tenant isolation
 - 56.permissions
 - 57.notifications
-

PHASE 10 — HARDENING

- 58.Redis isolation
 - 59.WebSocket isolation
 - 60.File isolation
 - 61.Search isolation
 - 62.Export isolation
 - 63.Analytics isolation
 - 64.AI isolation
-

PHASE 11 — SECURITY

- 65.IDOR tests
 - 66.Cross-tenant tests
 - 67.Cross-scope tests
 - 68.Role escalation tests
 - 69.Permission escalation tests
 - 70.Impersonation tests
-

75. DEFINITION OF DONE

Aucune fonctionnalité n'est considérée comme terminée tant que :

- ☐ Backend implémenté
 - ☐ DB/migration implémentée
 - ☐ API fonctionnelle
 - ☐ Authentication
 - ☐ Authorization
 - ☐ Tenant isolation
 - ☐ Scope isolation
 - ☐ IDOR testé
 - ☐ Frontend fonctionnel
 - ☐ Mobile fonctionnel si concerné
 - ☐ Offline sécurisé si concerné
 - ☐ Redis sécurisé
 - ☐ fichiers sécurisés
 - ☐ notifications sécurisées
 - ☐ tests automatisés
 - ☐ documentation
 - ☐ build réussi
 - ☐ application démarrée
 - ☐ parcours utilisateur réellement testé
 - ☐ aucun mock
 - ☐ aucun TODO critique
 - ☐ commit Git
-

76. FORMAT DE RAPPORT APRÈS CHAQUE ÉTAPE

Après chaque slice, produire :

IMPLEMENTATION REPORT

Fonctionnalité

Nom.

Objectif

Description.

Architecture

Description des changements.

Database

Tables/migrations.

Backend

Services/repositories/controllers.

API

Endpoints.

Security

Authentication.

Authorization.

Tenant isolation.

Scope.

IDOR tests.

Frontend

Pages/components.

Mobile

Screens/providers/cache.

Tests

Tests exécutés.

Validation

Résultat du parcours réel.

Fichiers modifiés

Liste complète.

Git

Commit.

Prochaine étape

Étape suivante.

77. RÈGLES DE SÉCURITÉ NON NÉGOCIABLES

NE JAMAIS considérer :

`tenantId` envoyé par frontend

comme preuve d'appartenance.

NE JAMAIS considérer :

`churchId`

comme preuve d'autorisation.

NE JAMAIS considérer :

`role` envoyé par frontend

comme fiable.

NE JAMAIS considérer :

`bouton` caché

comme sécurité.

NE JAMAIS considérer :

URL difficile à deviner

comme sécurité.

Toute autorisation doit être vérifiée côté serveur.

78. RÈGLE D'OR

Chaque requête métier doit être conceptuellement validée ainsi :

WHO ARE YOU?

+

WHICH TENANT?

+

WHICH MEMBERSHIP?

+

WHICH ROLE?

+

WHICH PERMISSION?

+

WHICH SCOPE?

+

WHICH RESOURCE?

+

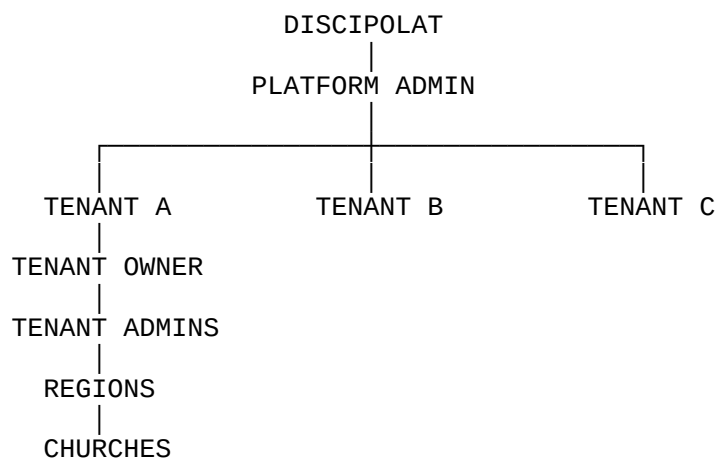
DO YOU OWN / CONTROL THIS RESOURCE?

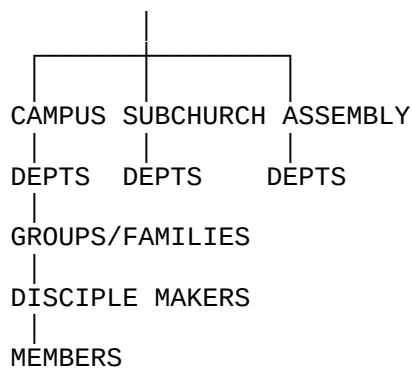
Si une seule condition échoue :

ACCESS DENIED

79. ARCHITECTURE FINALE ATTENDUE

Le résultat final doit permettre :





Chaque niveau possède :

- son rôle ;
- ses permissions ;
- son scope ;
- ses données accessibles ;
- ses limites ;
- ses interfaces adaptées.

80. OBJECTIF BUSINESS

Cette architecture doit permettre à Discipolat de devenir un véritable SaaS.

Une nouvelle organisation doit pouvoir :

Créer son compte



Créer son tenant



Personnaliser Discipolat



Créer son organisation



Créer ses églises



Créer ses sous-églises



Créer ses départements



Créer ses groupes



Inviter ses responsables



Importer ses membres



Utiliser Discipolat

sans intervention manuelle d'un développeur Discipolat.

81. OBJECTIF FINAL

Le système doit être suffisamment flexible pour permettre :

PETITE ÉGLISE

1 tenant
1 église
3 départements
100 membres

mais également :

GRANDE ORGANISATION

1 tenant
20 régions
300 églises
1 000 campus
5 000 départements
500 000 membres

sans modifier fondamentalement l'architecture.

82. PREMIÈRE MISSION DE L'AGENT

Commence maintenant.

NE CODE RIEN.

Effectue uniquement :

AUDIT MULTI-TENANT COMPLET

Analyse le repository existant.

Inspecte :

- backend ;
- frontend ;
- mobile ;
- PostgreSQL ;
- Flyway ;

- Spring Security ;
- JWT ;
- rôles ;
- permissions ;
- repositories ;
- services ;
- controllers ;
- Redis ;
- WebSocket ;
- notifications ;
- fichiers ;
- cache local ;
- tests.

Identifie toutes les données qui doivent devenir tenant-aware.

Identifie toutes les routes qui nécessitent un contrôle de scope.

Identifie toutes les méthodes repository susceptibles de créer un IDOR.

Identifie toutes les données actuellement globales.

Puis produis :

1. ARCHITECTURE ACTUELLE
2. FAILLES MULTI-TENANT
3. FAILLES RBAC
4. FAILLES IDOR
5. MODÈLE DE DONNÉES ACTUEL
6. MODÈLE DE DONNÉES CIBLE
7. MODÈLE DES RÔLES
8. MODÈLE DES PERMISSIONS
9. MODÈLE DES SCOPES
10. HIÉRARCHIE ORGANISATIONNELLE
11. STRATÉGIE DE MIGRATION
12. API À MODIFIER

13. FRONTEND À MODIFIER

14. MOBILE À MODIFIER

15. REDIS À MODIFIER

16. WEBSOCKET À MODIFIER

17. TESTS À CRÉER

18. RISQUES

19. PLAN D'IMPLÉMENTATION

20. PREMIÈRE SLICE À DÉVELOPPER

NE MODIFIE AUCUN FICHIER avant d'avoir terminé cet audit.

Après l'audit, attends la validation du responsable du projet avant de commencer la première modification architecturale majeure.

FIN DU PROMPT